TP EXPRESS.JS

October 2024

IID3

SUPERVISED BY

Prof. OURDOU Amal

PREPARED BY

NAJI Ez zoubir

1.Express.js:

- Express.js is a minimal and flexible Node.js web application framework.
- It provides a robust set of features for building web and mobile applications.
- With Express.js, you can create web applications, RESTful APIs, and single-page applications.
- It simplifies server-side development by offering routing and middleware capabilities.
- Express.js allows for easy integration with databases and supports various templating engines.

2.**Middlewares in Express.js:**

- Middlewares are functions that have access to the request, response, and next middleware function.
- They can execute code, modify request and response objects, end the request-response cycle, or call the next middleware.

Exemple 1

```javascript
const express = require('express');
const app = express();

// Middleware to log a message for each incoming request
function simpleLogger(req, res, next) {
    console.log('A request has been received');
    // Pass control to the next middleware or route handler
    next();
}

// Apply the middleware to all routes
app.use(simpleLogger);

app.get('/', (req, res) => {
    res.send('Home page');
});

app.listen(3000, () => {
    console.log('Server is running on port 3000');
});
```

- Imports the Express.js module.
- Creates an Express application.
- Defines a middleware function simpleLogger that logs a message to the console for each incoming request.
- Applies the middleware to all routes.
- Defines a GET endpoint / that sends the message "Home page" as a response.
- Starts the server listening on port 3000.

It displays the message "Home page" in a web browser, indicating that the application is running successfully and responding to requests. The URL localhost:3000 in the browser address bar confirms that you are accessing the application running locally on your machine.

C:\Users\naji\Desktop\IID3_TPS\JS Mobile\TP2_Express.js>node exemple_Middlewares.js: This indicates the command that was executed to start the Node.js application.
Server is running on port 3000: This confirms that the server is listening on port 3000, which means it is ready to accept requests.
A request has been received: This message indicates that a request has been received by the server.

Exemple 2

- Imports the Express.js module.
- Creates an Express application.
- Defines a middleware function addWelcomeMessage that adds a "Welcome!" message to the beginning of the response.
- Applies the middleware to all routes.
- Defines a GET endpoint / that sends the message "This is the home page." as a response.
- Starts the server listening on port 3000.

```javascript
const express = require('express');
const app = express();

// Middleware to add a welcome message to the response
function addWelcomeMessage(req, res, next) {
    // Replace the original send method to add a message
    const originalSend = res.send;
    res.send = function (body) {
        body = 'Welcome! ' + body; // Adds 'Welcome!' to the beginning of the response
        originalSend.call(this, body); // Sends the modified response
    };
    next();
}

// Add the middleware for all routes
app.use(addWelcomeMessage);

app.get('/', (req, res) => {
    res.send('This is the home page.');
});

app.listen(3000, () => {
    console.log('Server is running on port 3000');
});
```

Creating a simple CRUD application

1. Create a Project Directory
2. Initialize a Node.js Project

```
JS  Mobile  cd TP2_Express.js
TP2_Express.js  npm init -y
Wrote to C:\Users\naji\Desktop\IID3_TPs\JS  Mobile\TP2_Express.js\package.json:

{
  "name": "tp2_express",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

3. Install Express

```
TP2_Express.js  npm install express

added 65 packages, and audited 66 packages in 3s

13 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

4. Set Up Express: run the server using app.listen

```
// --- Q4: Set Up Express ---

const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to parse JSON in request bodies
app.use(express.json());

// Array to store our items (in-memory database)
let items = [];
```

```
PROBLEMS    OUTPUT    TERMINAL    PORTS

TERMINAL

TP2_Express.js  node app.js
Server started on http://localhost:3000
```

Sets up a basic Express.js web application. It includes a middleware to parse JSON data from requests and initializes an empty array to store items in memory, simulating a simple database.

5. Create a POST Endpoint: This will allow us to add items to a local variable

```javascript
// --- Q5: Create a POST Endpoint ---

// This endpoint allows adding new items
app.post('/items', (req, res) => {
    // Create a new item object with an auto-incremented ID
    const newItem = {
        id: items.length + 1,
        name: req.body.name,
        description: req.body.description
    };
    // Add the new item to our array
    items.push(newItem);
    // Respond with the created item and a 201 (Created) status
    res.status(201).json(newItem);
});
```

Defines a POST endpoint /items that allows adding new items to the in-memory database. When a POST request is made to this endpoint, the code extracts the name and description from the request body, creates a new item object with an auto-incremented ID, adds the item to the items array, and responds with the created item and a 201 status code.

6. Create a GET Endpoint: This will allow us to retrieve all items.

```
// --- Q6: Create a GET Endpoint ---

// This endpoint retrieves all items
app.get('/items', (req, res) => {
    // Simply return the entire items array
    res.json(items);
});

```

Defines a GET endpoint /items that retrieves all items stored in the items array. When a GET request is made to this endpoint, the code simply returns the entire items array as a JSON response.

7. Create a GET Endpoint by ID: This will allow us to get a specific item.

```
// --- Q7: Create a GET Endpoint by ID ---

// This endpoint retrieves a specific item by its ID
app.get('/items/:id', (req, res) => {
    // Parse the ID from the URL parameter
    const id = parseInt(req.params.id);
    // Find the item in our array
    const item = items.find(item => item.id === id);
    if (item) {
        // If found, return the item
        res.json(item);
    } else {
        // If not found, return a 404 error
        res.status(404).json({ message: "Item not found" });
    }
});
```

Defines a GET endpoint /items/:id that retrieves a specific item by its ID. When a GET request is made to this endpoint with an ID parameter, the code parses the ID from the URL, finds the corresponding item in the items array, and returns the item as a JSON response if found. If the item is not found, it returns a 404 error.

8. Create a PUT Endpoint: This will allow us to update an existing item

```
// --- Q8: Create a PUT Endpoint ---

// This endpoint updates an existing item
app.put('/items/:id', (req, res) => {
    // Parse the ID from the URL parameter
    const id = parseInt(req.params.id);
    // Find the index of the item in our array
    const itemIndex = items.findIndex(item => item.id === id);
    if (itemIndex > -1) {
        // If found, update the item by merging existing data with new data
        const updatedItem = { ...items[itemIndex], ...req.body };
        items[itemIndex] = updatedItem;
        // Return the updated item
        res.json(updatedItem);
    } else {
        // If not found, return a 404 error
        res.status(404).json({ message: "Item not found" });
    }
});
```

Defines a PUT endpoint /items/:id that updates an existing item by its ID. When a PUT request is made to this endpoint with an ID parameter, the code parses the ID from the URL, finds the index of the item in the items array, updates the item by merging existing data with new data from the request body, and returns the updated item. If the item is not found, it returns a 404 error.

9. Create a DELETE Endpoint: This will allow us to delete an item

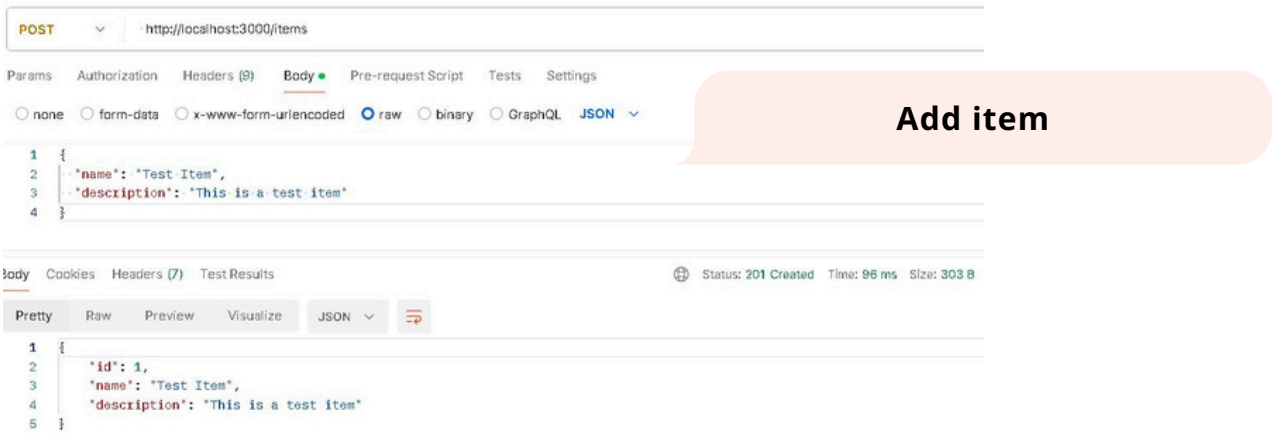
```
// --- Q9: Create a DELETE Endpoint ---

// This endpoint deletes an item
app.delete('/items/:id', (req, res) => {
    // Parse the ID from the URL parameter
    const id = parseInt(req.params.id);
    // Find the index of the item in our array
    const itemIndex = items.findIndex(item => item.id === id);
    if (itemIndex > -1) {
        // If found, remove the item from the array
        items.splice(itemIndex, 1);
        // Return a 204 (No Content) status to indicate successful deletion
        res.status(204).send();
    } else {
        // If not found, return a 404 error
        res.status(404).json({ message: "Item not found" });
    }
});
```

Defines a DELETE endpoint /items/:id that deletes an existing item by its ID. When a DELETE request is made to this endpoint with an ID parameter, the code parses the ID from the URL, finds the index of the item in the items array, removes the item from the array, and returns a 204 status code to indicate successful deletion. If the item is not found, it returns a 404 error.

10. Start the Server: In your terminal, run the server.

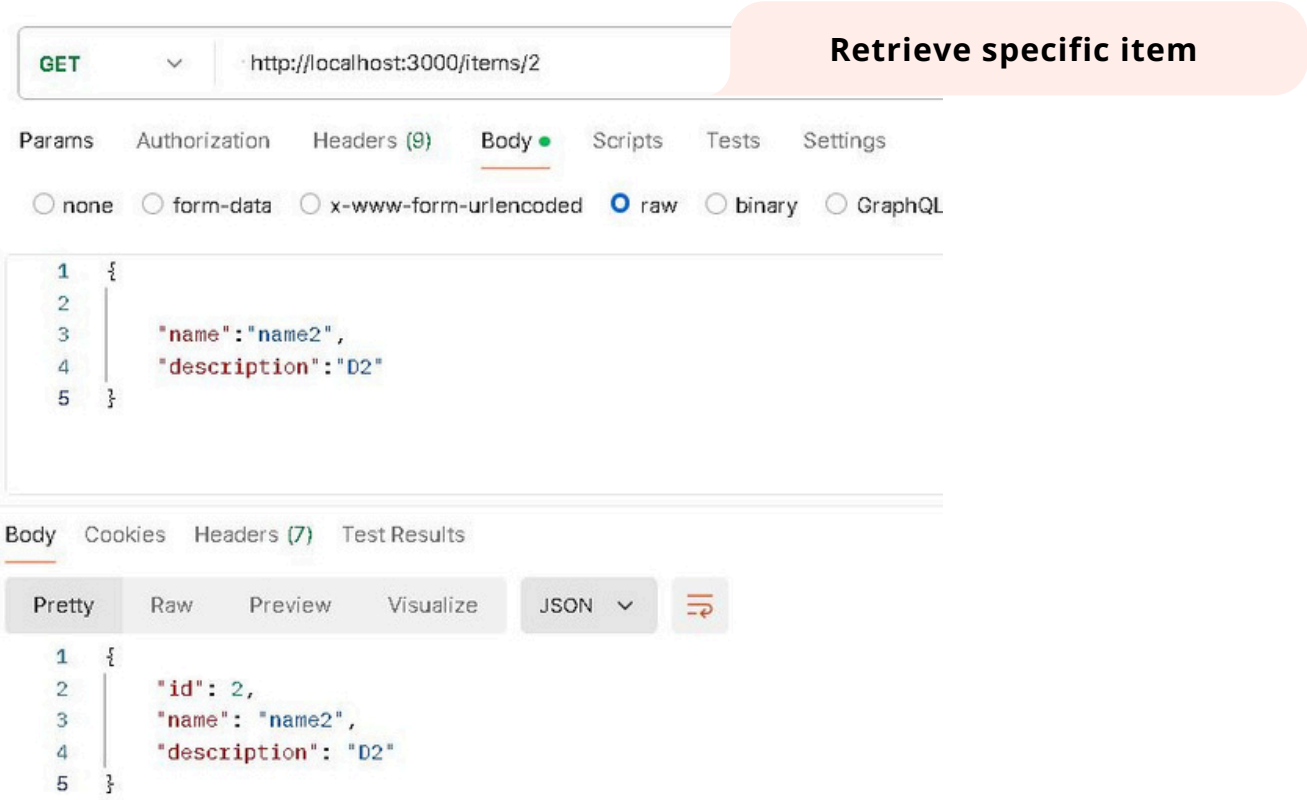
```
// Start the server
app.listen(PORT, () => {
    console.log(`Server started on http://localhost:${PORT}`);
});
```

11. Test the Endpoints using Postman



Successful POST request to the endpoint /items on the server http://localhost:3000. The request body contains a JSON object with the properties name and description, which are used to create a new item. The server responds with a 201 Created status code and returns the created item as a JSON object with an additional id property. This indicates that the item was successfully added to the database.

Successful GET request to the endpoint /items on the server http://localhost:3000. The server responded with a 200 OK status code and returned an array of all items as a JSON response. This indicates that the GET request was successful and that the server retrieved all the items stored in the database.



shows a successful GET request to the endpoint /items/2 on the server http://localhost:3000. This request is specifically retrieving the item with ID 2 from the database. The server responds with a 200 OK status code and returns the item as a JSON object, indicating that the item was found and successfully retrieved.

Update an existing item

PUT http://localhost:3000/items/2

Params Authorization Headers (9) Body Pre-request Script Tests

none form-data x-www-form-urlencoded raw binary GraphQ

```
1 {
2   "name": "Updated",
3   "description": "Updated from 2 to 3"
4 }
```

Body Cookies Headers (7) Test Results

Pretty Raw Preview Visualize JSON

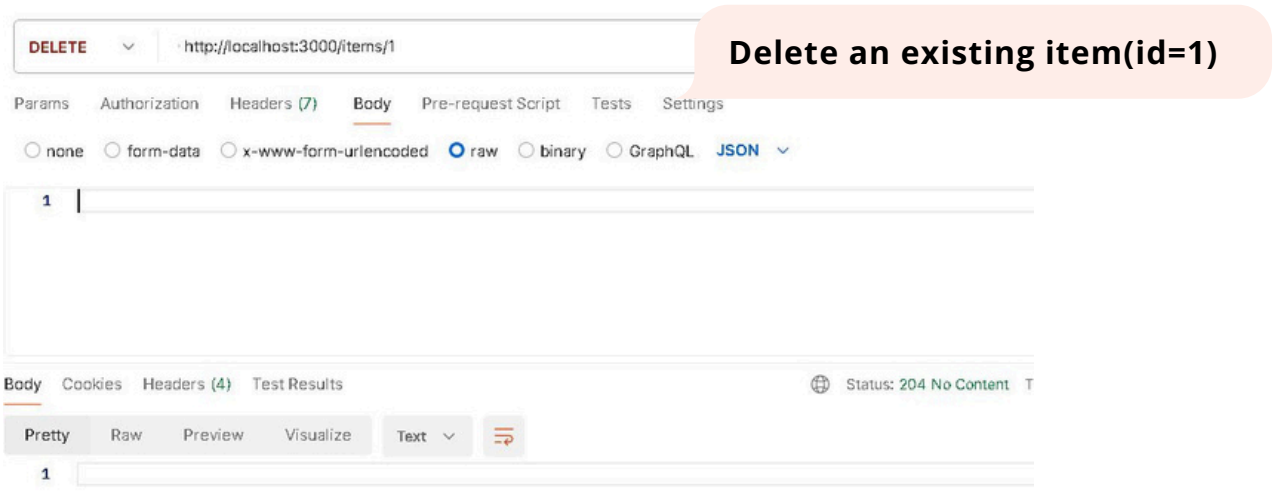
```
1 {
2     "id": 2,
3     "name": "Updated",
4     "description": "Updated from 2 to 3"
5 }
```

Successful PUT request to the endpoint /items/2 on the server http://localhost:3000. The request body contains a JSON object with the properties name and description, which are used to update the item with ID 2. The server responds with a 200 OK status code and returns the updated item as a JSON object. This indicates that the item was successfully updated in the database.

Body Cookies Headers (7) Test Results

Pretty Raw Preview Visualize JSON

```
1  [
2      {
3          "id": 1,
4          "name": "Test Item",
5          "description": "This is a test item"
6      },
7      {
8          "id": 2,
9          "name": "Updated",
10         "description": "Updated from 2 to 3"
11     }
12 ]
```



Shows a successful DELETE request to the endpoint /items/1 on the server http://localhost:3000. The server responded with a 204 No Content status code, indicating that the item with ID 1 was successfully deleted from the database.

Thank you